Notes on Computer Class Exercises ST451 2025/2026

London School of Economics and Political Science

Computer Class 1

Activity 1. Assume that the posterior of the variable of interest θ, $\pi(\theta|y)$ is the Gamma$(2, 1)$ distribution. Provide the answers to the questions below using Monte Carlo. Verify by contrasting with the exact answers where possible.

(a) Provide the Bayes estimate for θ.

(b) Give a 95% credible interval for θ.

(c) Calculate the posterior variance, $E[\theta^3|y]$ and $P(\theta > 3|y)$.

(d) Give a kernel density plot for its pdf.

Activity 2. Assume that we observe 1 success in a Binomial experiment involving 7 trials. Assign Uniform$(0, 1)$ prior on the probability of success θ and derive its posterior. Then, provide the answers to the below using Monte Carlo. Verify by contrasting with the exact answers where possible.

(a) Provide the Bayes estimate for θ.

(b) Give a 95% credible interval for θ.

(c) Calculate the posterior variance, $E[1/\theta|y]$ and $P(\theta < 0.2|y)$.

(d) Give a kernel density plot for its pdf.

Activity 3. A bank manager wants to decide on whether she should employ additional staff in the cashiers of their branch. This is usually determined by the number of customers visiting their branch on an average day. **Sample**: An experiment is conducted and the number of customers visiting the branch is recorded on 10 random days and are shown below:

$$X = (86, 98, 77, 94, 107, 108, 93, 72, 89, 81)$$

(a) Split the data into a training set (first 5 observations) and a test set (remaining observations).

(b) Assign a $\mathcal{N}(\mu, 100)$ model and an improper prior to the training set. This gives a predictive distribution of the $\mathcal{N}(\bar{X}, \frac{100}{n} + 100)$. Use it to obtain point forecasts for the test set. Use both Frequentist and Bayesian inference and compare their performance.

(c) Repeat the previous step but with 95% prediction intervals rather than point forecasts.

(d) For the frequentist approach one can estimate the mean μ via the MLE, $\hat{\mu}$ which is the sample mean (of the training data) as with the Bayesian approach (only because we used an improper prior). The distribution for the future data can then be the $\mathcal{N}(\bar{X}, 100)$.

(e) The MSE can be used to assess the forecasts. For the prediction intervals we can just count the number of test data contained in them.

(f) Discuss the differences or similarities between the frequentist and Bayesian approaches.

Activity 1. Assume that the posterior of the variable of interest θ, $\pi(\theta|y)$ is the Gamma$(2,1)$ distribution. Provide the answers to the questions below using Monte Carlo. Verify by contrasting with the exact answers where possible.

(a) Provide the Bayes estimate for θ.

(b) Give a 95% credible interval for θ.

(c) Calculate the posterior variance, $E[\theta^3|y]$ and $P(\theta > 3|y)$.

(d) Give a kernel density plot for its pdf.

(a) We can provide the median of θ under $\pi(\theta|y)$ as Bayes estimator. Recall the Gamma density with (shape, scale)$= (\alpha, \beta) = (2,1)$ parametrisation (which, in general, is the one in `numpy.random.gamma`; check out the `scipy.stats.gamma` parametrisation):

$$\begin{aligned}\pi(\theta|y) &= \frac{1}{\Gamma(\alpha)\beta^\alpha}\theta^{\alpha-1}e^{-\theta/\beta} \\ &= \theta e^{-\theta},\ \theta \geq 0\end{aligned}$$

The CDF is then

$$F_{\theta|y}(z) = \int_0^z \theta e^{-\theta} d\theta = 1 - (z+1)e^{-z}$$

which is continuous, strictly increasing (a bijection of $[0,\infty), [0,1)$) so the quantile function is

$$\Gamma_{2,1}(u) := F^{-1}_{\theta|y}(u) = \{u \in (0,1) : (z+1)e^{-z} = 1-u\}$$

We use `scipy.stats.gamma.ppf(q=u,a=2,loc=0,scale=1)` (note that $\beta = 1$ makes the parametrisation choice indifferent) to find the (almost exact) quantile value for given `u` $\in (0,1)$.

(b) The 95% credible interval for the median is then $[\Gamma_{2,1}(0.025), \Gamma_{2,1}(0.975)]$ because:

$$\begin{aligned}&P(\theta \in [\Gamma_{2,1}(0.025), \Gamma_{2,1}(0.975)]|y) \\ &= F_{\theta|y}(\Gamma_{2,1}(0.975)) - F_{\theta|y}(\Gamma_{2,1}(0.025)) \\ &= 0.975 - 0.025 = 0.95\end{aligned}$$

(c) We have

$$V[\theta|y] = \alpha\beta^2 = 2, \quad E[\theta^3|y] = \int_0^\infty \theta^4 e^{-\theta} d\theta = \Gamma(5) = 24$$
$$P(\theta > 3|y) = 1 - F_{\theta|y}(3) = 4e^{-3} \approx 0.1991$$

(d) Given a sample $X = (X_1, ..., X_n)$ of length n drawn from $\pi(\theta|y)$, the KDE is

$$f(x) = \frac{1}{n}\sum_{1\leq k\leq n} K(x - X_k),\ x \in \mathbb{R}$$

for kernel $K \geq 0$ with $\|K\|_1 = 1$; in `scipy.stats.gaussian_kde` it is a normal density.

Activity 2. Assume that we observe 1 success in a Binomial experiment involving 7 trials. Assign Uniform$(0,1)$ prior on the probability of success θ and derive its posterior. Then, provide the answers to the below using Monte Carlo. Verify by contrasting with the exact answers where possible.

(a) Provide the Bayes estimate for θ.

(b) Give a 95% credible interval for θ.

(c) Calculate the posterior variance, $E[1/\theta|y]$ and $P(\theta < 0.2|y)$.

(d) Give a kernel density plot for its pdf.

(a) We derive the general posterior given uniform prior on θ given a vector $y \in \{0,1\}^n$ with average $\overline{y}$:

$$\begin{aligned} f(\theta|y) &\propto f(y|\theta)f(\theta) \\ &= \binom{n}{n\overline{y}}\theta^{n\overline{y}}(1-\theta)^{n(1-\overline{y})} \cdot \mathbf{1}_{[0,1]}(\theta) \end{aligned}$$

Hence $f(\theta|y)$ is a Beta density with $(\alpha, \beta) = (n\overline{y}+1, n+1-n\overline{y})$ because

$$\begin{aligned} \binom{n}{n\overline{y}}\int_{[0,1]}\theta^{n\overline{y}}(1-\theta)^{n(1-\overline{y})}d\theta &= \binom{n}{n\overline{y}}\frac{\Gamma(n\overline{y}+1)\Gamma(n+1-n\overline{y})}{\Gamma(n+2)} \\ &= \binom{n}{n\overline{y}}\frac{(n\overline{y})!(n(1-\overline{y}))!}{(n+1)!} = \frac{1}{n+1} \end{aligned}$$

So

$$f(\theta|y) = \frac{(n+1)!}{(n\overline{y})!(n(1-\overline{y}))!}\theta^{n\overline{y}}(1-\theta)^{n(1-\overline{y})}$$

Here we have $(n, n\overline{y}) = (7, 1)$ so $(\alpha, \beta) = (2, 7)$. We can find the quantile at `u` $\in (0,1)$ with `scipy.stats.beta.ppf(q=u,a=2,b=7)`. We can provide the median of θ under $f(\theta|y)$ as Bayes estimator.

(b) If $B_{2,7}(u)$ is the quantile function of Beta$(2,7)$ and $F_{\theta|y}$ is its CDF, we see that

$$\begin{aligned} &P(\theta \in [B_{2,7}(0.025), B_{2,7}(0.975)]|y) \\ &= F_{\theta|y}(B_{2,7}(0.975)) - F_{\theta|y}(B_{2,7}(0.025)) \\ &= 0.975 - 0.025 = 0.95 \end{aligned}$$

(c) We get

$$\begin{aligned} V[\theta|y] &= \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)} = \frac{14}{810} \\ E[1/\theta|y] &= \frac{(n+1)!}{(n\overline{y})!(n(1-\overline{y}))!}\int_{[0,1]}\theta^{n\overline{y}-1}(1-\theta)^{n(1-\overline{y})}d\theta = \frac{n+1}{n\overline{y}} = 8 \end{aligned}$$

For $P(\theta < 0.2|y)$, we may use `scipy.stats.beta.cdf(x=0.2,a=2,b=7)`.

(d) Again, we use `scipy.stats.gaussian_kde` for a Gaussian KDE.

Activity 3. A bank manager wants to decide on whether she should employ additional staff in the cashiers of their branch. This is usually determined by the number of customers visiting their branch on an average day. **Sample**: An experiment is conducted and the number of customers visiting the branch is recorded on 10 random days and are shown below:

$$X = (86, 98, 77, 94, 107, 108, 93, 72, 89, 81)$$

(a) Split the data into a training set (first 5 observations) and a test set (remaining observations).

(b) Assign a $\mathcal{N}(\mu, 100)$ model and an improper prior to the training set. This gives a predictive distribution of the $\mathcal{N}(\bar{X}, \frac{100}{n} + 100)$. Use it to obtain point forecasts for the test set. Use both Frequentist and Bayesian inference and compare their performance.

(c) Repeat the previous step but with 95% prediction intervals rather than point forecasts.

(d) For the frequentist approach one can estimate the mean μ via the MLE, $\hat{\mu}$ which is the sample mean (of the training data) as with the Bayesian approach (only because we used an improper prior). The distribution for the future data can then be the $\mathcal{N}(\bar{X}, 100)$.

(e) The MSE can be used to assess the forecasts. For the prediction intervals we can just count the number of test data contained in them.

(f) Discuss the differences or similarities between the frequentist and Bayesian approaches.

(b), (Assign a $\mathcal{N}(\mu, 100)$ model and an improper prior to the training set. This gives a predictive distribution of the $\mathcal{N}(\bar{X}, \frac{100}{n} + 100)$)

The improper prior for μ in $\mathcal{N}(\mu, \sigma^2)$ with σ^2 known is the constant $f(\mu) = 1, \mu \in \mathbb{R}$. We get for $y = (y_1, ..., y_n) \in \mathbb{R}^n$ with average $\overline{y} = \frac{1}{n} y'1$:

$$\begin{aligned}
f(\mu|y) &\propto f(y|\mu) f(\mu) \\
&\propto \exp\left(-\frac{1}{2\sigma^2} \sum_{1 \le k \le n} (y_k - \mu)^2 \right) \\
&= \exp\left(-\frac{1}{2\sigma^2} \left(n\mu^2 + \sum_{1 \le k \le n} y_k^2 - 2\mu n \overline{y} \right) \right) \\
&= \exp\left(-\frac{1}{2\sigma^2} \left(n\mu^2 + n\overline{y}^2 - 2\mu n \overline{y} \right) \right) \exp\left(-\frac{1}{2\sigma^2} \left(\sum_{1 \le k \le n} y_k^2 - n\overline{y} \right) \right) \\
&\propto \exp\left(-\frac{(\mu - \overline{y})^2}{2\sigma^2 (\frac{1}{n})} \right)
\end{aligned}$$

hence the posterior is $\mathcal{N}(\overline{y}, \sigma^2/n)$. Recall that the posterior predictive for $y_{n+1} \in \mathbb{R}$ is

$$f(y_{n+1}|y) = \int_{\mathbb{R}} f(y_{n+1}|\mu) f(\mu|y) d\mu$$

We thus get the posterior predictive

$$\begin{aligned} f(y_{n+1}|y) &= \int_{\mathbb{R}} f(y_{n+1}|\mu) f(\mu|y) d\mu \\ &\propto \int_{\mathbb{R}} \exp\bigg(- \frac{(y_{n+1}-\mu)^2}{2\sigma^2} - \frac{(\mu-\overline{y})^2}{2\sigma^2(\frac{1}{n})} \bigg) d\mu \end{aligned}$$

This is a Gaussian convolution. Indeed, write

$$u(\nu) := \int_{\mathbb{R}} \exp\bigg(- \frac{(\nu-(\mu-\overline{y}))^2}{2\sigma^2} - \frac{(\mu-\overline{y})^2}{2\sigma^2(\frac{1}{n})} \bigg) d\mu, \; \nu \in \mathbb{R}$$

It can be proved (sketch: substitute $x = \mu - \overline{y}$, complete the squares, use Gaussian integral) that

$$u(\nu) \propto \exp\bigg(- \frac{\nu^2}{2\sigma^2(1+\frac{1}{n})} \bigg)$$

Therefore

$$\begin{aligned} &\int_{\mathbb{R}} \exp\bigg(- \frac{(y_{n+1}-\mu)^2}{2\sigma^2} - \frac{(\mu-\overline{y})^2}{2\sigma^2(\frac{1}{n})} \bigg) d\mu \\ &= \int_{\mathbb{R}} \exp\bigg(- \frac{((y_{n+1}-\overline{y})-(\mu-\overline{y}))^2}{2\sigma^2} - \frac{(\mu-\overline{y})^2}{2\sigma^2(\frac{1}{n})} \bigg) d\mu \\ &= u(y_{n+1}-\overline{y}) \\ &\propto \exp\bigg(- \frac{(y_{n+1}-\overline{y})^2}{2\sigma^2(1+\frac{1}{n})} \bigg) \end{aligned}$$

But then

$$f(y_{n+1}|y) = \frac{1}{\sqrt{2\pi\sigma^2(1+\frac{1}{n})}} \exp\bigg(- \frac{(y_{n+1}-\overline{y})^2}{2\sigma^2(1+\frac{1}{n})} \bigg)$$

That is

$$X_{n+1}|(X_1,...,X_n) = y \sim \mathcal{N}\bigg(\overline{y}, \sigma^2\bigg(1+\frac{1}{n}\bigg)\bigg)$$

as we wanted to show.

Computer Class 2

Linear Regression / Polynomial Fitting

We start with the polynomial curve fitting exercise seen in the lecture slides. The *truth* is the function $\sin(2\pi x)$ for $x \in [0, 1]$. We observe 10 points, equispaced in $[0, 1]$ - array `x`, with $\mathcal{N}(0, 0.3^2)$ independent measurement error, array `y`. The array `xg` contains a grid of 100 points in $[0, 1]$.

Activity 1. Fit polynomials of 3rd and 9th degrees to the previous data and provide their coefficients and their training mean squared error.

Ridge and Lasso Regression

We will use all powers of `x` up to 9 but we also include a penalty term on the squared value of the coefficients. The amount of penalisation is controlled by the parameter '`lam`'. We will repeat the exercise under a more realistic setting:

(a) Generate more 100 noisy observations from the function $\sin(2\pi x)$ as before.

(b) Randomly select 10 of those. These will form the **training sample** that will be used to fit the models and obtain the regression coefficients.

(c) Set aside the remaining 90 observations. These will be the **test sample** that will be used to assess the predictions of each model that was estimated in the train sample.

(d) Contrasting those predictions to the real data will allow us to estimate the **test error** and assess the out of sample performance of each model.

(e) Monitor both the training and test errors for each model.

Activity 2. Find the order of polynomial that produces the smallest test MSE. Can you beat that with Ridge or Lasso Regression? In other words can you find penalty '`lam`' such that the Ridge or Lasso regression model produces an even smaller test MSE?

Real data example

Finally we will look at a real data example (`diabetes.data.txt`), some info on which is given below:

(a) 442 diabetes patients;

(b) 10 main variables, `x_1,...,x_10` : age, gender, body mass index, average blood pressure (map), and six blood serum measurements (tc, ldl, hdl, tch, ltg, glu);

(c) 45 interactions of the form `xjxk`;

(d) 9 quadratic effects of the form `x_j2`;

(e) Measure of disease progression taken one year later, `y`.

Activity 3. Fit and calculate the test error of the Ridge and Lasso regression models. Try to find an optimal `lam` for them.

Linear Regression / Polynomial Fitting

We start with the polynomial curve fitting exercise seen in the lecture slides. The *truth* is the function $\sin(2\pi x)$ for $x \in [0,1]$. We observe 10 points, equispaced in $[0,1]$ - array `x`, with $\mathcal{N}(0, 0.3^2)$ independent measurement error, array `y`. The array `xg` contains a grid of 100 points in $[0,1]$.

Activity 1. Fit polynomials of 3rd and 9th degrees to the previous data and provide their coefficients and their training mean squared error.

We have a model of noisy observations $Y = (Y_1, ..., Y_n)$ of the form

$$\begin{bmatrix} Y_1 \\ Y_2 \\ Y_3 \\ ... \\ Y_n \end{bmatrix} = \begin{bmatrix} \sin(2\pi x_1) \\ \sin(2\pi x_2) \\ \sin(2\pi x_3) \\ ... \\ \sin(2\pi x_n) \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ ... \\ \varepsilon_n \end{bmatrix}, \quad \varepsilon_k \overset{\text{IID}}{\sim} \mathcal{N}(0, 0.3^2)$$

(this is array `y`) where

$$(x_1, x_2, x_3, ..., x_n) = \left(0, \frac{1}{n-1}, \frac{2}{n-1}, ..., \frac{n-2}{n-1}, 1\right)$$

and we set $n = 10$ (this is array `x`). Let $k \in \mathbb{N}_0$ and define the Vandermonde matrix

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 & ... & x_1^k \\ 1 & x_2 & x_2^2 & x_2^3 & ... & x_2^k \\ 1 & x_3 & x_3^2 & x_3^3 & ... & x_3^k \\ ... & ... & ... & ... & ... & \\ 1 & x_n & x_n^2 & x_n^3 & ... & x_n^k \end{bmatrix}$$

and we find (with `sklearn.linear_model.LinearRegression`) $\hat{\beta} \in \mathbb{R}^{k+1}$ which solves

$$\hat{\beta} = \text{argmin}_{\beta \in \mathbb{R}^{k+1}} \|Y - X\beta\|_2^2$$

which is solved in closed form by the OLS argument (if $X'X$ is invertible):

$$\begin{aligned} \frac{d}{d\beta}\|Y - X\beta\|_2^2 &= \frac{d}{d\beta}(Y - X\beta)'(Y - X\beta) \\ &= \frac{d}{d\beta}(Y'Y - 2X'Y\beta + \beta'X'X\beta) \\ &= -2X'Y + (X'X + (X'X)')\beta \\ &= -2X'Y + 2X'X\beta \\ \implies \hat{\beta} &= (X'X)^{-1}X'Y \end{aligned}$$

The fitted values and the training MSE are respectively

$$\hat{Y} = X\hat{\beta}, \quad \text{MSE} = \frac{1}{n}\|Y - \hat{Y}\|_2^2$$

Ridge and Lasso Regression

We will use all powers of `x` up to 9 but we also include a penalty term on the squared value of the coefficients. The amount of penalisation is controlled by the parameter '`lam`'. We will repeat the exercise under a more realistic setting:

(a) Generate more 100 noisy observations from the function $\sin(2\pi x)$ as before.

(b) Randomly select 10 of those. These will form the **training sample** that will be used to fit the models and obtain the regression coefficients.

(c) Set aside the remaining 90 observations. These will be the **test sample** that will be used to assess the predictions of each model that was estimated in the train sample.

(d) Contrasting those predictions to the real data will allow us to estimate the **test error** and assess the out of sample performance of each model.

(e) Monitor both the training and test errors for each model.

Activity 2. Find the order of polynomial that produces the smallest test MSE. Can you beat that with Ridge or Lasso Regression? In other words can you find penalty '`lam`' such that the Ridge or Lasso regression model produces an even smaller test MSE?

Given $\lambda > 0$ (this is `lam`), regression matrix X and observations Y, the Ridge regression coefficient found by the regression model

```
sklearn.linear_model.Ridge(alpha=lam)
```

solves

$$\hat{\beta}^{\text{Ridge}} = \text{argmin}_{\beta \in \mathbb{R}^{k+1}}(\|Y - X\beta\|_2^2 + \lambda\|\beta\|_2^2)$$

which is found in closed form by the OLS argument (if $X'X + \lambda I_k$ is invertible):

$$\begin{aligned}
\frac{d}{d\beta}(\|Y - X\beta\|_2^2 + \lambda\|\beta\|_2^2) &= \frac{d}{d\beta}((Y - X\beta)'(Y - X\beta) + \lambda\beta'\beta) \\
&= \frac{d}{d\beta}(Y'Y - 2\tilde{X}'Y\beta + \beta'X'X\beta + \lambda\beta'\beta) \\
&= -2X'Y + (X'X + (X'X)')\beta + 2\lambda\beta \\
&= -2X'Y + 2X'X\beta + 2\lambda\beta \\
\implies \hat{\beta}^{\text{Ridge}} &= (X'X + \lambda I_k)^{-1}X'Y
\end{aligned}$$

Given $\lambda > 0$ (again, this is `lam`) the LASSO regression coefficient found by the regression model

```
sklearn.linear_model.Lasso(alpha=lam)
```

solves

$$\hat{\beta}^{\text{LASSO}} = \text{argmin}_{\beta \in \mathbb{R}^{k+1}}\left(\|Y - X\beta\|_2^2 + \lambda\|\beta\|_1\right)$$

where m is the dimension of Y. Unlike $\hat{\beta}^{\text{Ridge}}$, the estimate $\hat{\beta}^{\text{LASSO}}$ is necessarily found numerically. In both cases, fitted values and MSE are found as in polynomial regression.

Computer Class 3

Automobile Bodily Injury Claims Data

The data are automobile injury claims data using data from the Insurance Research Council (IRC), a division of the American Institute for Chartered Property Casualty Underwriters and the Insurance Institute of America. The data, collected in 2002, contains information on the gender of the claimant, attorney involvement, years of driving experience and the economic loss (`LOSS`, in thousands). A detailed description of the variables in the data is provided below:

(a) `Attorney`: Whether the claimant is represented by an attorney (= 1 if yes and = 0 if no);

(b) `CLMSEX`: Claimant's gender (= 1 if male and = 0 if female);

(c) `CLMAGE`: Claimant's age minus the age driving license was obtained;

(d) `LOSS`: The claimant's total economic loss (in thousands $).

For confidentiality issues we consider here a sample of simulated data similar to that of a state is considered. The data are in the provided file '`automobileBI.csv`'.

Activity 1. Obtain the linear regression MLEs and 95% confidence intervals for the model with the `LOSS` variable as response with only the `ATTORNEY` variable and without using any Python built in function. Check your answers against the `statsmodels` corresponding function (`statsmodels.regression.linear_model.OLS`).

Bayesian Linear Regression

We now turn to Bayesian inference. We start by calculating the posterior parameters that correspond to the unit information prior. For σ^2 we set the prior IGamma(a_0, b_0).

Activity 2. Consider all possible regression models with the `LOSS` variable as response and some (or all or none) of the other variables present (in their current form, no polynomials). There are 8 such models. Calculate the model evidence for each of them, report the optimal one and present posterior estimates of its coefficients.

Activity 3. Given that the scatter plots do not provide support for the linear model, consider transforming the target and/or the features and repeat.

Automobile Bodily Injury Claims Data

The data are automobile injury claims data using data from the Insurance Research Council (IRC), a division of the American Institute for Chartered Property Casualty Underwriters and the Insurance Institute of America. The data, collected in 2002, contains information on the gender of the claimant, attorney involvement, years of driving experience and the economic loss (`LOSS`, in thousands). A detailed description of the variables in the data is provided below:

(a) `Attorney`: Whether the claimant is represented by an attorney (= 1 if yes and = 0 if no);

(b) `CLMSEX`: Claimant's gender (= 1 if male and = 0 if female);

(c) `CLMAGE`: Claimant's age minus the age driving license was obtained;

(d) `LOSS`: The claimant's total economic loss (in thousands $).

For confidentiality issues we consider here a sample of simulated data similar to that of a state is considered. The data are in the provided file '`automobileBI.csv`'.

Activity 1. Obtain the linear regression MLEs and 95% confidence intervals for the model with the `LOSS` variable as response with only the `ATTORNEY` variable and without using any Python built in function. Check your answers against the `statsmodels` corresponding function (`statsmodels.regression.linear_model.OLS`).

Given a linear model of the form for the response $Y = (Y_1, Y_2, ..., Y_n)$

$$Y = X\beta + \varepsilon, \quad X = \begin{bmatrix} 1 & x_{1,1} & ... & x_{1,m} \\ 1 & x_{2,1} & ... & x_{2,m} \\ ... & ... & ... & ... \\ 1 & x_{n,1} & ... & x_{n,m} \end{bmatrix}, \quad \varepsilon \sim \mathcal{N}_n(0, \sigma^2 I_n)$$

where $\beta \in \mathbb{R}^m$, the log-likelihood for (β, σ^2) is

$$\ell(y; \beta, \sigma^2) = -\frac{n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|y - X\beta\|_2^2, \quad y \in \mathbb{R}^n$$

Given any $\sigma^2 > 0$, the second term is always maximised by the OLS estimate $\hat{\beta} = (X'X)^{-1}X'y$. On the other hand,

$$\frac{d}{d\sigma^2}\ell(y; \hat{\beta}, \sigma^2) = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4}\|y - X\hat{\beta}\|_2^2 = 0 \implies \hat{\sigma}^2 = \frac{1}{n}\|y - X\hat{\beta}\|_2^2$$

So the MLE is $(\hat{\beta}, \hat{\sigma}^2)$. Since Y is multivariate normal, we have that $\hat{\beta}$ is also multivariate normal. Thus

$$E[\hat{\beta}] = (X'X)^{-1}X'E[Y] = \beta$$
$$V[\hat{\beta}] = (X'X)^{-1}X'V[Y]X(X'X)^{-1} = \sigma^2(X'X)^{-1}$$

So we may choose to obtain marginal confidence intervals for $\hat{\beta}$ from $\mathcal{N}_m(\hat{\beta}, \hat{\sigma}^2(X'X)^{-1})$ (if $m \ll n$), or from a t-distribution argument (if m is close to n).

Bayesian Linear Regression

We now turn to Bayesian inference. We start by calculating the posterior parameters that correspond to the unit information prior. For σ^2 we set the prior IGamma(a_0, b_0).

Activity 2. Consider all possible regression models with the `LOSS` variable as response and some (or all or none) of the other variables present (in their current form, no polynomials). There are 8 such models. Calculate the model evidence for each of them, report the optimal one and present posterior estimates of its coefficients.

Given $\sigma^2 > 0$, the unit information prior is a multivariate normal prior distribution for β which depends on X. If $X'X$ is invertible, it is specified by (Murphy, p. 236)

$$\begin{aligned} \Omega_0 &:= n(X'X)^{-1} \\ \mu_0 &\in \mathbb{R}^m \\ \beta|X, \sigma^2 &\sim \mathcal{N}_m(\mu_0, \sigma^2\Omega_0) \end{aligned}$$

We specify an IGamma(a_0, b_0) as a prior for σ^2. It can be proved (tedious, but straightforward) that the joint posterior density is thus (Murphy, 7.69-7.73 p. 235)

$$\begin{aligned} f(\beta, \sigma^2|X, y) &= f(y|X, \beta, \sigma^2) f(\beta|X, \sigma^2) f(\sigma^2) \\ &\propto \frac{1}{(\sigma^2)^{m/2}} \exp\left(-\frac{1}{2\sigma^2}(\beta - \mu_n)'(X'X + \Omega_0^{-1})(\beta - \mu_n) \right) \\ &\cdot \frac{1}{(\sigma^2)^{\frac{n+2a_0}{2}+1}} \exp\left(-\frac{2b_0 + \|y\|_2^2 - \mu_n'\Omega_n^{-1}\mu_n + \mu_0'\Omega_0^{-1}\mu_0}{2\sigma^2} \right) \\ &\propto f(\beta|\sigma^2, X, y) f(\sigma^2|X, y) \end{aligned}$$

where

$$\begin{aligned} \beta|\sigma^2, X, y &\sim \mathcal{N}(\mu_n, \sigma^2\Omega_n) \\ \sigma^2|X, y &\sim \text{IGamma}(a_n, b_n) \end{aligned}$$

and

$$\begin{aligned} \mu_n &:= \Omega_n(X'y + \Omega_0^{-1}\mu_0) \\ \Omega_n &:= (X'X + \Omega_0^{-1})^{-1} \\ a_n &:= a_0 + \frac{n}{2} \\ b_n &:= b_0 + \frac{1}{2}(\|y\|_2^2 - \mu_n'\Omega_n^{-1}\mu_n + \mu_0'\Omega_0^{-1}\mu_0) \end{aligned}$$

The model evidence (which depends only on X) is

$$\begin{aligned} f(y|X) &= \frac{f(y|X, \beta, \sigma^2) f(\beta, \sigma^2|X)}{f(\beta, \sigma^2|X, y)} \\ \implies \ln f(y|X) &= \underbrace{\ln f(y|X, \beta, \sigma^2)}_{\text{log-likelihood}} + \underbrace{\ln f(\beta, \sigma^2|X)}_{\text{log-prior}} - \underbrace{\ln f(\beta, \sigma^2|X, y)}_{\text{log-posterior}} \end{aligned}$$

We can evaluate the rhs for admissible values of β, σ^2 (e.g. $(\mu_n, \frac{b_n}{a_n - 1})$ if $a_n > 1$, the respective means of the posteriors).

Computer Class 4

Breast Cancer Survival Data

The dataset contains cases from a study that was conducted between 1958 and 1970 at the University of Chicago's Billings Hospital on the survival of patients who had undergone surgery for breast cancer. Four columns:

(a) `age`: age of patient at time of operation (numerical);

(b) `year`: patient's year of operation (year - 1900, numerical);

(c) `nodes`: number of positive axillary nodes detected (numerical);

(d) `y`: survival status (class attribute): 1 = the patient survived 5 years or longer, 2 = the patient died within 5 years.

See more info at the UCI repository `https://archive.ics.uci.edu/ml/machine-learning-databases/haberman/haberman.names`.

Activity 1. Provide the ouput from the MLE logistic regression and Bayesian logistic regression (via Laplace approximation) approaches.

Activity 2. Consider 2 logistic regression models:

(a) $M1$: with covariates `age` and `nodes`.

(b) $M2$: with covariates `year`, `age` and `nodes`.

(a) Compare them using the log-evidence, the BIC, the area under the ROC curve and the log scoring rule. (b) Discuss the differences in predicted probabilities from MLE and Bayesian models.

Linear Discriminant Analysis

Compute the classification probability $P(Y = 1|X)$ by modeling the joint probability $P_{Y,X}$. That is called a generative model.

Activity 3. Fit logistic regression and LDA models on the training data and evaluate their predictive performance (ROC, area under the ROC, log score) on the test data. Use `year`, `age` and `nodes`.

Breast Cancer Survival Data

The dataset contains cases from a study that was conducted between 1958 and 1970 at the University of Chicago's Billings Hospital on the survival of patients who had undergone surgery for breast cancer. Four columns:

(a) `age`: age of patient at time of operation (numerical);

(b) `year`: patient's year of operation (year - 1900, numerical);

(c) `nodes`: number of positive axillary nodes detected (numerical);

(d) `y`: survival status (class attribute): 1 = the patient survived 5 years or longer, 2 = the patient died within 5 years.

See more info at the UCI repository `https://archive.ics.uci.edu/ml/machine-learning-databases/haberman/haberman.names`.

Activity 1. Provide the ouput from the MLE logistic regression and Bayesian logistic regression (via Laplace approximation) approaches.

Logistic regression is a generalised linear model (GLM) specialised to binary data $Y = (Y_1, ..., Y_n)$ with $P(Y \in \{0,1\}^n) = 1$ and regressor matrix $X \in \mathbb{R}^{n\times(m+1)}$

$$X = \begin{bmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{bmatrix} = \begin{bmatrix} 1 & x_{1,1} & ... & x_{1,m} \\ 1 & x_{2,1} & ... & x_{2,m} \\ ... & ... & ... & ... \\ 1 & x_{n,1} & ... & x_{n,m} \end{bmatrix}$$

The GLM is characterised by the logit link function g:

$$g(z) = \ln \frac{z}{1-z} \implies P(Y_k = 1|X) = g^{-1}(x_k \cdot \beta), \quad g^{-1}(y) = \frac{1}{1+e^{-y}}$$

MLE logistic regression (Murphy 8.3.1-3 pp. 246-250, Bishop 4.3.2-4.3.3 pp. 205-208). The likelihood is given by

$$L(y|X,\beta) = \prod_{1\le k\le n} (g^{-1}(x_k \cdot \beta))^{y_k}(1 - g^{-1}(x_k \cdot \beta))^{1-y_k}, \quad y \in \{0,1\}^n$$

and we aim to find $\beta = (\beta_0, ..., \beta_m)$ which minimises (resp. maximises) the negative log-likelihood $-\ell = -\ln L$ (resp. the log-likelihood $\ell = \ln L$). It can be shown that (recall x_k are row vectors):

$$\nabla(-\ell)(\beta) = \begin{bmatrix} \partial_{\beta_0}(-\ell)(\beta) \\ ... \\ \partial_{\beta_m}(-\ell)(\beta) \end{bmatrix} = \sum_{1\le k\le n} (g^{-1}(x_k \cdot \beta) - y_k)x_k'$$

$$H(-\ell)(\beta) = \begin{bmatrix} \partial_{\beta_0^2}(-\ell)(\beta) & ... & \partial_{\beta_0\beta_m}(-\ell)(\beta) \\ ... & ... & ... \\ \partial_{\beta_0\beta_m}(-\ell)(\beta) & ... & \partial_{\beta_m^2}(-\ell)(\beta) \end{bmatrix} = \sum_{1\le k\le n} g^{-1}(x_k \cdot \beta)(1 - g^{-1}(x_k \cdot \beta))x_k'x_k$$

It can also be shown that $H_\beta(-\ell)$ is convex, hence $-\ell$ has a unique global minimum. Since no closed form is known, a naïve algorithm of Newton-Raphson type to find a minimum may follow the pseudocode:

`Initialise` $\beta = \beta^{(0)}$.
`For` $k = 1, 2, \ldots$ `until convergence do:`

(a) `Evaluate` $\nabla(-\ell)(\beta^{(k-1)})$;
(b) `Evaluate` $H(-\ell)(\beta^{(k-1)})$;
(c) `Set` $\beta^{(k)} = \beta^{(k-1)} - (H(-\ell)(\beta^{(k-1)}))^{-1}\nabla(-\ell)(\beta^{(k-1)})$

where we need to specify a `convergence` condition. The estimate $\hat{\beta}$ can be chosen to be the value obtained by the algorithm, if convergence is reached. In general, the library `scikit-learn` provides the logistic regression class

`sklearn.linear_model.LogisticRegression`

which can use a variety of robust solver algorithms to estimate β.

Bayesian logistic regression (via Laplace approximation) (Murphy 8.4.1-4 pp. 255-260). Suppose that β^{M} is the (real) modal value of the posterior density of β, and that the posterior has the form

$$f(\beta|X,y) \propto \exp(-\mathcal{E}(\beta))$$
$$\mathcal{E}(\beta) \approx \mathcal{E}(\beta^{\mathrm{M}}) + (\beta - \beta^{\mathrm{M}})\underbrace{\nabla(\mathcal{E})(\beta^{\mathrm{M}})}_{=0} + \frac{1}{2}(\beta - \beta^{\mathrm{M}})' H(\mathcal{E})(\beta^{\mathrm{M}})(\beta - \beta^{\mathrm{M}})$$

so that we obtain the Laplace approximation

$$\beta|X,y \sim \mathcal{N}(\beta^{\mathrm{M}}, (H(\mathcal{E})(\beta^{\mathrm{M}}))^{-1})$$

So we may start out with a Gaussian prior $\beta \sim \mathcal{N}(\beta_0, \Sigma_0)$, obtaining

$$f(\beta|X,y) = L(y|X,\beta)f(\beta)$$

and we set the mean (and mode) of the Gaussian posterior as the MAP estimate

$$\hat{\beta}^{\mathrm{M}} := \mathrm{argmax}_\beta f(\beta|X,y)$$

and the precision matrix (inverse of variance-covariance matrix) as

$$\begin{aligned} H(\mathcal{E})(\hat{\beta}^{\mathrm{M}}) &= H(-\ln f(\beta|X,y))(\hat{\beta}^{\mathrm{M}}) \\ &= -H(\ln f(\beta))(\hat{\beta}^{\mathrm{M}}) + H(-\ln L(y|X,\beta))(\hat{\beta}^{\mathrm{M}}) \\ &= \Sigma_0^{-1} + H(-\ell)(\hat{\beta}^{\mathrm{M}}) \end{aligned}$$

Again, in order to find $\hat{\beta}^{\mathrm{M}}$ we can implement a naïve Newton-Raphson algorithm to minimise the negative log-likelihood.

Activity 2. Consider 2 logistic regression models:

(a) $M1$: with covariates `age` and `nodes`.

(b) $M2$: with covariates `year`, `age` and `nodes`.

(a) Compare them using the log-evidence, the BIC, the area under the ROC curve and the log scoring rule. (b) Discuss the differences in predicted probabilities from MLE and Bayesian models.

Log-evidence. (seen previously)

BIC (Bayesian information criterion). Defined as

$$\text{BIC} := \underbrace{\ell(\hat{\beta})}_{\text{log-likelihood at } \beta \text{ estimate}} - \underbrace{\frac{m}{2} \ln n}_{\text{penalty on number of regressors}}$$

As a model choice criterion, the higher BIC is preferable.

Area under the ROC (receiver operating characteristic) curve. A binary classifier determines the class of $Y \in \{0, 1\}$ given X by comparing $P(Y = 1|X)$ with a threshold τ, that is

$$\hat{Y} = 1 \iff P(Y = 1|X) > \tau$$

Given a test set, the true positive rate (TPR) and the false positive rate (FPR) given a threshold τ are defined as

$$\text{TPR}(\tau) := \frac{\text{correct classifications as class 1}}{\text{total observations with class 1}}$$

$$\text{FPR}(\tau) := \frac{\text{incorrect classifications as class 1}}{\text{total observations with class 0}}$$

The estimated ROC curve is the graph $\varphi(\tau) = (\text{FPR}(\tau), \text{TPR}(\tau))$ in $[0, 1]^2$ for some values of τ, and the area under the ROC curve is the estimated area of the unit square below the ROC curve. From `scikit-learn` we use

```
sklearn.metrics.roc_curve(y_true, y_score)
sklearn.metrics.roc_auc_score(y_true, y_score)
```

where `y_score` are the model predictions for `y_true`. As a model choice criterion, the higher area under the ROC curve (AUC) is preferable.

Log-scoring rule (or log-loss). Defined as

$$\text{LS}(Y, X) := -\ln \left(P(Y = 1|X)^Y (1 - P(Y = 1|X))^{1-Y} \right)$$

Given two models with respective $P(Y = 1|X)$, the lower LS is preferable. Given a test set, we use

```
sklearn.metrics.log_loss(y_true, y_pred)
```

where `y_pred` are the model predictions for `y_true`.

Linear Discriminant Analysis

Compute the classification probability $P(Y = 1|X)$ by modeling the joint probability $P_{Y,X}$. That is called a generative model.

Activity 3. Fit logistic regression and LDA models on the training data and evaluate their predictive performance (ROC, area under the ROC, log score) on the test data. Use `year`, `age` and `nodes`.

We shall use the classification model

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

which implements Gaussian linear discriminant analysis (Gaussian LDA) (Murphy 4.2.2-4.2.4 pp. 103-106). We thus assume a Bernoulli marginal with parameter π for the binary class Y and Gaussian conditional $X|Y$ of the regressor $X = (X_1, ..., X_m)$ which different means μ_0, μ_1 for the two classes and identical variance-covariance matrix Σ:

$$Y \sim \text{Bernoulli}(\pi)$$
$$X|Y = k \sim \mathcal{N}_m(\mu_k, \Sigma),\ k = 0, 1$$

which yields for $x \in \mathbb{R}^m$

$$\begin{aligned} P(Y = 1|X = x) &= \frac{f(x|Y = 1)\pi}{f(x|Y = 1)\pi + f(x|Y = 0)(1 - \pi)} \\ &= \frac{1}{1 + \frac{f(x|Y=0)P(Y=0)}{f(x|Y=1)P(Y=1)}} \end{aligned}$$

Now note:

$$\begin{aligned} &\frac{f(x|Y = 0)P(Y = 0)}{f(x|Y = 1)P(Y = 1)} \\ &= \frac{1 - \pi}{\pi} \exp\left(-\frac{1}{2}(x'\Sigma^{-1}x + \mu_0'\Sigma^{-1}\mu_0 - 2\mu_0'\Sigma^{-1}x) + \frac{1}{2}(x'\Sigma^{-1}x + \mu_1'\Sigma^{-1}\mu_1 - 2\mu_1'\Sigma^{-1}x)\right) \\ &= \exp\Bigg(-\underbrace{\left(\ln\frac{\pi}{1-\pi} + \frac{1}{2}(\mu_0'\Sigma^{-1}\mu_0 - \mu_1'\Sigma^{-1}\mu_1)\right)}_{=:C} - \underbrace{(\mu_1 - \mu_0)'\Sigma^{-1}}_{=:\beta'} x\Bigg) \\ &= \exp(-(C + \beta' x)) \end{aligned}$$

Hence:

$$P(Y = 1|X = x) = \frac{1}{1 + e^{-(C+\beta' x)}}$$

If we set `solver='lsqr'` then the estimates $\hat{\pi}, \hat{\mu}_0, \hat{\mu}_1, \hat{\Sigma}$ are the MLE estimates of the Gaussian LDA.

Computer Class 5

Mean Field Approximation

Activity 1. Let $Y = (Y_1, \ldots, Y_n)$ be independent Poisson(λ) observations. Assume that λ follows the Gamma$(2, \beta)$ distribution, where β follows the Exponential(1) distribution. The aim is to draw inference from the posterior $\pi(\theta|y)$, where $\theta = (\lambda, \beta)$.

The variational Bayes algorithm approximates $\pi(\theta|y)$ using the mean field approximation

$$q(\theta|y, \phi) = q(\lambda|y, \phi)q(\beta|y, \phi)$$

It can be shown that such an algorithm may consist of the following steps:

(a) Initialise at $q(\lambda)$ to be the Gamma(a_λ, b_λ) and $q(\beta)$ to be the Gamma(a_β, b_β) distribution, setting

$$a_\lambda = 2 + \sum_i Y_i, \quad b_\lambda = b_\lambda^0, \quad a_\beta = 3, \quad \text{and} \quad b_\beta = b_\beta^0.$$

(b) Iteratively update b_λ and b_β until the parameters or the ELBO converge. At iteration i will have:

(i). Set

$$b_\lambda^i = n + E_{q(\beta)}[\beta] = n + 3/b_\beta^{i-1}$$

(ii). Set

$$b_\beta^i = 1 + E_{q(\lambda)}[\lambda] = 1 + \left(2 + \sum_i Y_i\right)/b_\lambda^{i-1}$$

Task: Code the above algorithm and fit it to the simulated data. Check your answers in terms of the lambda estimates.

Automatic Differentiation Variational Inference (ADVI) with `PyMC`

Demonstration in Logistic (Binomial) Regression

$$\mu_k = \beta_0 + \beta_1 x_k, \; x_k \in [-10, 20]$$
$$p_k = \frac{1}{1 + e^{-\mu_k}}$$
$$Y_k \sim \text{Binomial}(n, p_k), \;\; \text{independent}$$

Activity 2. In order to incorporate a deterministic function in the `PyMC` model definition, you can use the function below (for the sigmoid function)

```
p = pymc.Deterministic("p", pm.math.invlogit(mu))
```

Also, the Bernoulli likelihood can be specified with the code below (with `n`, `p` and `observed` to be provided by the user):

```
pymc.Binomial("y", n=..., p=..., observed= ...)
```

Mean Field Approximation

Activity 1. Let $Y = (Y_1, \ldots, Y_n)$ be independent Poisson(λ) observations. Assume that λ follows the Gamma$(2, \beta)$ distribution, where β follows the Exponential(1) distribution. The aim is to draw inference from the posterior $\pi(\theta|y)$, where $\theta = (\lambda, \beta)$.

The variational Bayes algorithm approximates $\pi(\theta|y)$ using the mean field approximation

$$q(\theta|y, \phi) = q(\lambda|y, \phi)q(\beta|y, \phi)$$

It can be shown that such an algorithm may consist of the following steps: (...)

(shape-rate Gamma!) The log-joint distribution is characterised by the log-density

$$\begin{aligned}\ln f(y, \lambda, \beta) &= \ln\left(\prod_{1\le k\le n} \frac{e^{-\lambda}\lambda^{y_k}}{y_k!}\right) + \ln\left(\frac{\beta^2}{\Gamma(2)}\lambda e^{-\beta\lambda}\right) + \ln(e^{-\beta}) \\ &= \left(-n\lambda + \ln\lambda \sum_{1\le k\le n} y_k\right) + (2\ln\beta + \ln\lambda - \lambda\beta) + (-\beta) + C^{(1)}\end{aligned}$$

For $\ln q(\lambda)$, we focus on the terms involving λ:

$$\begin{aligned}&\ln q(\lambda) = -(n + E_{q(\beta)}[\beta])\lambda + \ln\lambda\left(\sum_{1\le k\le n} y_k + 1\right) + C^{(2)} \\ \implies\ & q(\lambda) \propto e^{-\lambda(n+E_{q(\beta)}[\beta])}\lambda^{\sum_{1\le k\le n} y_k + 1} \\ \implies\ & \lambda \sim \text{Gamma}\left(2 + \sum_{1\le k\le n} y_k, n + E_{q(\beta)}[\beta]\right)\end{aligned}$$

while for $\ln q(\beta)$ we focus on the terms involving β:

$$\begin{aligned}&\ln q(\beta) = 2\ln\beta - \beta(1 + E_{q(\lambda)}[\lambda]) + C^{(3)} \\ \implies\ & q(\beta) \propto e^{-\beta(1+E_{q(\lambda)}[\lambda])}\beta^2 \\ \implies\ & \beta \sim \text{Gamma}\left(3, 1 + E_{q(\lambda)}[\lambda]\right)\end{aligned}$$

If we initialise at $q(\lambda)$ to be the Gamma(a_λ, b_λ) and $q(\beta)$ to be the Gamma(a_β, b_β) distribution, with

$$a_\lambda = 2 + \sum_{1\le k\le n} y_k, \quad b_\lambda = b_\lambda^0, \quad a_\beta = 3, \quad \text{and} \quad b_\beta = b_\beta^0$$

We get the desired recursion (recall that the Gamma average is shape/rate):

$$\begin{aligned}b_\lambda^i &= n + E_{q(\lambda)}[\beta] = n + \frac{3}{b_\beta^{i-1}} \\ b_\beta^i &= 1 + E_{q(\beta)}[\lambda] = 1 + \frac{2 + \sum_{1\le k\le n} y_k}{b_\lambda^i}\end{aligned}$$

Computer Class 6

Load the Iris dataset

`sklearn.datasets.load_iris()` consists of 3 different types of irises' (Setosa, Versicolor, and Virginica) petal and sepal length, stored in a 150x4 `numpy.ndarray`. The rows being the samples and the columns being: `sepal length (cm)`, `sepal width (cm)`, `petal length (cm)` and `petal width (cm)`.

Fitting GMMs using the EM algorithm

`sklearn.mixture.GaussianMixture` does not necesarily allocates individuals with certainty but with probabilities. We need to fit models with different numbers of clusters and different type of covariance matrices to identify the best one. This is done via the BIC (the smaller the better in this case). Types of covariance matrices:

(a) `spherical`: each cluster k has covariance $\sigma_k^2 I$;

(b) `tied`: full covariance matrix but the same across clusters;

(c) `diag`: diagonal covariance matrix, different for each cluster;

(d) `full`: full covariance matrix, different for each cluster.

We consider $k = 1, ..., 8$ and four types of covariance matrix.

Activity 1. Repeat the analysis using only two of the four variables. Do we get a different conclusion on the number of clusters?

Simulate data to test the method

We will simulate data from a Gaussian mixture with three components and spherical covariance matrix. Generate random sample, three components:

(a) Mean $(0, 0)$, variance-covariance matrix I;

(b) Mean $(-6, 3)$, variance-covariance matrix $0.16I$;

(c) Mean $(3, -4)$, variance-covariance matrix $9I$.

We repeat the previous model search procedure to the data. We would like to test whether the optimal models will indeed be the one with three components and spherical covariance.

Activity 2. Conduct another simulation experiment generating data from a Gaussian mixture. Choose your own number of components, means and covariances.

Overfitted (Bayesian) Gaussian Mixtures

We will explore what happens when we fit a model with more components than the ones in the data. We will also apply the fully Bayesian model to the same data with a Dirichlet prior on the cluster probabilities with a low hyperparameter `weight_concentration_prior` of 0.01.

Activity 3. Fit the fully Bayesian approach to the Iris dataset and check the resulting number of clusters.

Load the Iris dataset

`sklearn.datasets.load_iris()` consists of 3 different types of irises' (Setosa, Versicolor, and Virginica) petal and sepal length, stored in a 150x4 `numpy.ndarray`. The rows being the samples and the columns being: `sepal length (cm)`, `sepal width (cm)`, `petal length (cm)` and `petal width (cm)`.

Fitting GMMs using the EM algorithm

`sklearn.mixture.GaussianMixture` does not necesarily allocates individuals with certainty but with probabilities. We need to fit models with different numbers of clusters and different type of covariance matrices to identify the best one. (...)

We set a number of k clusters. The Gaussian Mixture Model is specified by a mixture distribution. This means that an observation from a GMM with k clusters can be generated as following:

(a) Produce a draw $Z \sim \text{Multinomial}(1, \pi)$ where $\pi = (\pi_1, ..., \pi_k), \sum_\ell \pi_\ell = 1$ are the frequencies associated to each cluster. This determines the cluster Z.

(b) Once the cluster Z is drawn, draw the observation Y from the distribution corresponding to the cluster. In GMM, the distributions are all (d-dimensional) Gaussian:

$$Y|Z = k \sim \mathcal{N}_d(\mu_k, \Sigma_k)$$

where we set $\theta_k = (\mu_k, \Sigma_k)$.

(c) Repeat the procedure to generate another observation.

Note that a key point is that Z is unobserved. The joint likelihood for $y \in \mathbb{R}^d, z \in \{1, ..., k\}$ (a single datum) is

$$f(y, z|\pi, \theta_1, ..., \theta_k) = \prod_{1 \le \ell \le k} (f(y|\theta_\ell)\pi_\ell)^{\delta_\ell(z)}, \quad \delta_\ell(z) = \begin{cases} 1 & \text{if } z = \ell \\ 0 & \text{if } z \neq \ell \end{cases}$$

so the joint likelihood for $y \in \mathbb{R}^{d \times n}, z \in \{1, ..., k\}^n$ (n data) is

$$f(y_1, ..., y_n, z_1, ..., z_n|\pi, \theta_1, ..., \theta_k) = \prod_{1 \le m \le n} \prod_{1 \le \ell \le k} (f(y_m|\theta_\ell)\pi_\ell)^{\delta_\ell(z_m)}$$

The marginal of y for a single datum (y, z) is

$$f(y|\pi, \theta_1, ..., \theta_k) = \sum_{1 \le u \le k} f(y, u|\pi, \theta_1, ..., \theta_k) = \sum_{1 \le u \le k} f(y|\theta_u)\pi_u$$

and the joint will be the product $\prod_{1 \le m \le n} f(y_m|\pi, \theta_1, ..., \theta_k)$. The Expectation-Maximisation (EM) algorithm to estimate $(\pi, \theta) = (\pi_1, ..., \pi_k, \theta_1, ..., \theta_k)$ follows two steps. We initiate the algorithm with (π_0, θ_0), then we update with

(a) Expectation step: we find (by Bayes' theorem)

$$\gamma_{m,\ell} = P(Z_m = \ell|y_m, \pi_0, \theta_0) \propto \pi_{0,\ell} f(y_m|\theta_{0,\ell})$$

which are called responsibilities. The we write the expected (under the law $Z_m \sim \gamma$) joint log-likelihood:

$$\begin{aligned} Q(\pi, \theta, \pi_0, \theta_0) &= E_{Z\sim\gamma}\Big[\sum_{1\le m\le n}\sum_{1\le \ell\le k}\delta_\ell(Z_m)(\ln \pi_\ell + f(y_m|\theta_k))\Big] \\ &= \sum_{1\le m\le n}\sum_{1\le \ell\le k} E_{Z\sim\gamma}[\delta_\ell(Z_m)](\ln \pi_\ell + f(y_m|\theta_k)) \\ &= \sum_{1\le m\le n}\sum_{1\le \ell\le k} P(Z_m = \ell|y_m, \pi_0, \theta_0)(\ln \pi_\ell + f(y_m|\theta_k)) \\ &= \sum_{1\le m\le n}\sum_{1\le \ell\le k} \gamma_{m,\ell}(\ln \pi_\ell + f(y_m|\theta_k)) \end{aligned}$$

(b) Maximisation step: we set

$$(\pi_1, \theta_1) = \text{argmax}_{(\pi,\theta)} Q(\pi, \theta, \pi_0, \theta_0), \quad \text{s.t.} \quad 1'\pi = 1$$

where the constrained maximisation is performed e.g. with a Lagrangian. In GMM, we get for each $\ell \in \{1, ..., k\}$

$$\begin{aligned} \pi_{1,\ell} &= \frac{1}{n}\sum_{1\le m\le n}\gamma_{m,\ell} \\ \mu_{1,\ell} &= \frac{\sum_{1\le m\le n}\gamma_{m,\ell}y_m}{\sum_{1\le m\le n}\gamma_{m,\ell}} \\ \Sigma_{1,\ell} &= \frac{n}{\sum_{1\le m\le n}\gamma_{m,\ell}}\sum_{1\le m\le n}\gamma_{m,\ell}(y_m - \mu_{1,\ell})(y_m - \mu_{1,\ell})' \end{aligned}$$

(c) Repeat until convergence: if a convergence criterion is not reached, re-initialise with (π_1, θ_1) instead of (π_0, θ_0).

The result is called soft-allocation, since GMM then associates to each datum y_m the final $\gamma_{m,1}, ..., \gamma_{m,k}$, that is the probabilities that y_m belongs to each cluster. For this model, we will use the class

`sklearn.mixture.GaussianMixture`

where in particular we specify `n_components` and `covariance_type`. We can then `fit` the model to the given data, and then obtain the cluster classification probabilities for each observation with `predict_proba`.

Overfitted (Bayesian) Gaussian Mixtures

We will explore what happens when we fit a model with more components than the ones in the data. We will also apply the fully Bayesian model to the same data with a Dirichlet prior on the cluster probabilities with a low hyperparameter `weight_concentration_prior` of 0.01.

Activity 3. Fit the fully Bayesian approach to the Iris dataset and check the resulting number of clusters.

A fully Bayesian approach will take also π as random, as well as the means $\mu = (\mu_1, ..., \mu_k)$ and variance-covariance matrices (now we reparametrise with the precision matrices $\Lambda = (\Lambda_1, ..., \Lambda_k)$) associated to each cluster. In particular, we can specify that for a single observation $Y : \Omega \to \mathbb{R}^d$ we have the hierarchy

$$\begin{aligned} Y|Z = \ell, \mu, \Lambda &\sim \mathcal{N}_d(\mu_\ell, \Lambda_\ell^{-1}) \\ Z|\pi &\sim \text{Multinomial}(1, \pi) \\ \mu_\ell|\Lambda_\ell &\sim \mathcal{N}_d(m_0, (\beta_0\Lambda_\ell)^{-1}) \\ \Lambda_\ell &\sim \text{Wishart}(W_0, \nu_0, d) \\ \pi &\sim \text{Dirichlet}(\alpha_0 I_k, k) \end{aligned}$$

where $(m_0, \beta_0, W_0, \nu_0, \alpha_0)$ are prior hyperparameters. Recall that:

(a) Wishart(W_0, ν_0, d) is essentially a multivariate extension of the Gamma distribution, yielding a random $d \times d$ positive-definite matrix;

(b) Dirichlet(α_0, k) yields a random k-dimensional vector on the $k-1$ simplex (i.e. a random probability mass function on $\{1, 2, ..., k\}$) where $\alpha_0 I_k$ is a vector of lenght k with all entries equal to a specified α_0.

(c) If α_0 is small, the total mass of π will be concentrated in fewer entries of the vector (edges of the simplex). If α_0 is large, the total mass of π will be dispersed over the vector (center of the simplex).

The class

`sklearn.mixture.BayesianGaussianMixture`

is a fully Bayesian GMM where, in particular, `weight_concentration_prior` refers to the concentration prior α_0 of the Dirichlet prior, and `n_components` is k. We can then `fit` the model to the given data, and then obtain the cluster classification probabilities for each observation with `predict_proba`.